

Natural-Language Mathematical Statements to Formal Statements

Autoformalization as alignment, not translation

XMUM GE – Week 8

Ma Jiajun – May 26, 2026

Part I. Opening Case Study

One symbol, many formal objects

One Symbol, Many Formal Objects: Integral

On the board only this:

$$\int_a^b f(x) dx$$

Question: to write this as a Lean theorem statement, what is missing?

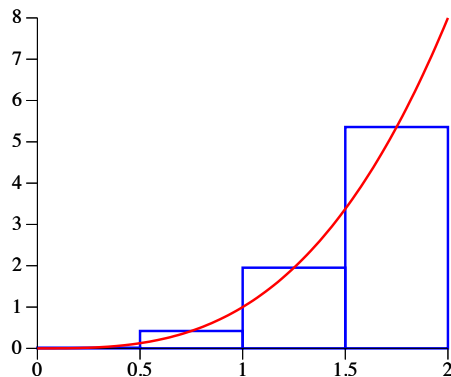
The missing piece is not Lean syntax. It is the mathematical object.

Possible readings

1. Riemann integral on a compact interval.
2. Lebesgue / Bochner integral wrt a measure.
3. Oriented interval integral $\int_a^b$.
4. Set integral over $[a, b]$, $(a, b]$, ...
5. Variable upper limit $x \mapsto \int_a^x f(t) dt$.
6. Antiderivative relation $F' = f$.

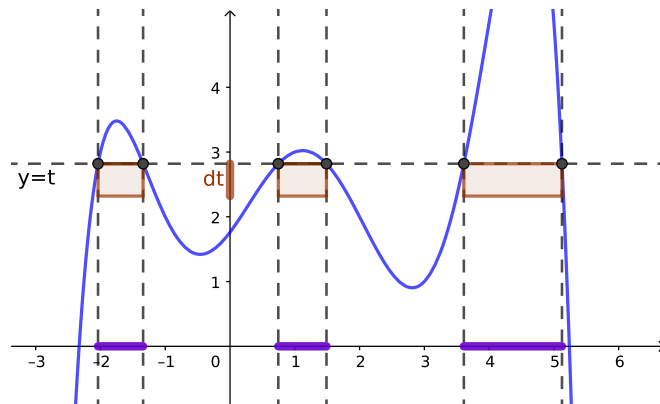
Riemann vs Lebesgue: Two Ways To Sum The Area

Riemann – slice the x -axis



$$\int f \approx \sum_i f(x_i) \Delta x_i$$

Lebesgue – slice the y -axis



$$\int f \approx \sum_j y_j \mu(\{x : f(x) \geq y_j\})$$

Riemann cuts the domain into vertical strips (height $\times$ width). **Lebesgue** cuts the range into horizontal layers (value $\times$ measure of level set). **Next slide's rule:** Lebesgue requires $\int |f| < \infty$ – conditionally convergent $\Rightarrow$ Lebesgue refuses, improper Riemann still accepts.

Mathlib Says "As $R \rightarrow \infty$ " With Filter and Tendsto

Natural sentence

$$\int_1^R \frac{\sin x}{x} dx \rightarrow L \quad \text{as } R \rightarrow \infty$$

Mathlib writes

```
Tendsto (fun R => ∫ x in 1..R, sin x / x) atTop (ℵ L)
```

A function, a *source filter*, a *target filter*. No ε - δ on the surface.

Filter = "eventually"

- `atTop : Filter ℝ` – R is arbitrarily large
- `ℵ L : Filter ℝ` – value lies in any neighborhood of L

`Tendsto f F G` reads: *f* pushes "eventually in F " to "eventually in G ".

Same shape, different filters: `Tendsto u atTop atTop` = sequence to ∞ . `Tendsto f (ℵ a) (ℵ b)` = f continuous at a with $f(a)=b$. ε - δ lives inside the filters; the **statement** uses filter algebra.

Running Example: Improper Riemann Exists, Lebesgue Does Not

$$\int_1^{\infty} \frac{\sin x}{x} dx \text{ exists.}$$

Reading A – improper Riemann integral: **true**

```
∃ L : ℝ,  
  Tendsto  
    (fun R : ℝ ⇒ ∫ x in (1:ℝ)..R, Real.sin x / x)  
    atTop  
    (ℳ L)
```

Finite interval integrals have a limit as $R \rightarrow \infty$.

Reading B – Lebesgue integrable on $[1, \infty)$: **false**

```
¬ IntegrableOn  
  (fun x : ℝ ⇒ Real.sin x / x)  
  (Set.Ici 1)
```

$\int_1^{\infty} |\sin x/x| dx = \infty$. Lebesgue integrability needs absolute integrability; this is only conditional convergence.

Putnam 2025 B2: Geometry Becomes Integral Ratios

$f : [0, 1] \rightarrow [0, \infty)$ strictly increasing continuous.

R = region under $y = f(x)$. $x_1, x_2 = x$ -centroids of R and of the solid of revolution. Prove $x_1 < x_2$.

Formal statement does not define R , centroid, or solid of revolution – it uses the analytic formulas directly.

```
theorem putnam_2025_b2
  (f : ℝ → ℝ)
  (hf_cont : ContinuousOn f (Icc 0 1))
  (hf_mono : StrictMonoOn f (Icc 0 1))
  (hf_nonneg : ∀ x ∈ Icc (0 : ℝ) 1, 0 ≤ f x) :
  (∫ x in (0:ℝ)..1, x * f x) /
    (∫ x in (0:ℝ)..1, f x) <
  (∫ x in (0:ℝ)..1, x * (f x) ^ 2) /
    (∫ x in (0:ℝ)..1, (f x) ^ 2) := by
  sorry
```

Geometric narrative replaced by analytically equivalent integral ratios. The π cancels; denominator positivity is a proof obligation, not a hypothesis.

Gemini Draft: Correct Shape, Lean Syntax Fixes

Two layers on this slide

1. Apply Lean syntax fixes from earlier sessions:

`import Mathlib, open Set Real`

`MeasureTheory Interval, noncomputable
section, Real.pi`, parameterize over `f`.

2. Once it typechecks, Gemini's **mathematical shape** is right.

```
noncomputable section
```

```
def area_R (f : ℝ → ℝ) : ℝ :=  
  ∫ x in (0:ℝ)..1, f x
```

```
def x1 (f : ℝ → ℝ) : ℝ :=  
  (∫ x in (0:ℝ)..1, x * f x) / area_R f
```

```
def vol_solid (f : ℝ → ℝ) : ℝ :=  
  Real.pi * ∫ x in (0:ℝ)..1, (f x)^2
```

```
def x2 (f : ℝ → ℝ) : ℝ :=  
  (Real.pi * ∫ x in (0:ℝ)..1, x * (f x)^2) /  
  vol_solid f
```

Both versions express the same analytic comparison: AxiomMath inlines the integrals; Gemini names them. Presentation choice, not error.

"Indefinite Integral" Is Not A Defined Object

(A) Derivative side

```
HasDerivAt : ( $\mathbb{k} \rightarrow F$ )  $\rightarrow$   $F \rightarrow \mathbb{k} \rightarrow \text{Prop}$   
-- predicate:  $F' = f \ x \text{ at } x$   
deriv : ( $\mathbb{k} \rightarrow F$ )  $\rightarrow \mathbb{k} \rightarrow F$   
-- total function; junk value 0 if no deriv
```

In

`Mathlib.Analysis.Calculus.Deriv.Basic`.
`HasDerivAt` is a `Prop`, `deriv` is a value.

(B) Definite integral side

```
intervalIntegral  
  : ( $\mathbb{R} \rightarrow E$ )  $\rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \text{Measure } \mathbb{R} \rightarrow E$   
-- notation:  $\int x \text{ in } a..b, f \ x \ d\mu$ 
```

In

`Mathlib.MeasureTheory.Integral.IntervalIntegral`.
built on Bochner `MeasureTheory.integral`.

There is **no** `indefiniteIntegral` / `antiderivative` / `primitive` def in `mathlib`. Any "indefinite integral" must reduce to (A) or (B); FTC is the theorem connecting them, not a definition.

Three Informal Phrases, Two Primitives

Natural language	Real identity	Mathlib expression
" F is an antiderivative of f "	predicate on side (A)	$\forall x, \text{HasDerivAt } F (f\ x) x$
"the antiderivatives of f "	set carved out by that predicate	$\{F \mid \forall x, \text{HasDerivAt } F (f\ x) x\}$
" $\int_a^x f(t) dt$ "	a function on side (B)	$\text{fun } x \Rightarrow \int t \text{ in } a..x, f\ t$

The first row uses **only** the derivative side. There is no integral in it.

The second row is a **set-builder**, not a `def`. mathlib has no `antiderivatives` definition.

The third row uses **only** the integral side. It is just a lambda over `intervalIntegral`.

Every alignment of "the indefinite integral of f " has to commit to one of three forms above – and therefore to one of the two primitives (A) or (B).

`deriv f x = 0` Is **Not** Evidence Of Differentiability

Core warning. `deriv f x = 0` does **not** imply that f is differentiable at x . The equation is **not** evidence of differentiability.

Why

`deriv` is a **total function**: Lean requires a value at every point.

If f is not differentiable at x , then `deriv f x` returns `0` by convention (junk value).

Two interpretations of `deriv f x = 0`

1. f is differentiable at x , derivative happens to be 0; **or**
2. f is *not* differentiable at x , `deriv` returned the default 0.

You cannot tell which from the equation alone.

Correct shape for " $f'(0) = 0$ ": use `HasDerivAt f 0 0` (packs existence + value together), or hold `DifferentiableAt ℝ f 0` as a separate hypothesis before reading off `deriv`. This is exactly why side (A)

Heaviside Step: The Junk-Value In Action

Setup

$$H(x) = \mathbf{1}_{[0,\infty)}(x)$$

```
def H : ℝ → ℝ :=  
  Set.indicator (Set.Ici 0) (fun _ => 1)
```

H is **not continuous** at 0, so certainly not differentiable there.

Yet in Lean

```
example : deriv H 0 = 0 := by sorry
```

This holds – but via the **junk-value** path, not because the derivative equals 0.

```
deriv H 0 = 0 ✓ (true)  
DifferentiableAt ℝ H 0 ✗ (false)  
HasDerivAt H 0 0 ✗ (false)
```

Reading `deriv f x = 0` as "the derivative at x is 0" is wrong whenever the function is not differentiable there. The compiler accepts the equation; the mathematical content has silently dropped out.

Fundamental Theorem Of Calculus As Statement Shapes

FTC-1

If $F(x) = \int_a^x f(t) dt$, then $F'(x) = f(x)$.

```
HasDerivAt
  (fun x => ∫ t in a..x, f t) (f x) x
```

FTC-2

If $F'(x) = f(x)$, then $\int_a^b f(x) dx = F(b) - F(a)$.

```
(∫ x in a..b, f' x) = F b - F a
```

Mathlib theorem names:

```
intervalIntegral.integral_eq_sub_of_hasDerivAt
intervalIntegral.integral_deriv_eq_sub'
```

The real content lives in the statement

- `HasDerivAt`, `HasDerivWithinAt`, or `deriv`?
- Where does the derivative hold?
- Integrability hypothesis?
- Oriented interval?
- Codomain structure?

"The fundamental theorem of calculus" $\implies$ a family of theorem schemas.

Worked Example: $f' \equiv 0 \implies f$ Constant (1/2)

Theorem (informal). If f is continuous on $[a, b]$ and $f' = 0$ on (a, b) , then f is constant on $[a, b]$.

The "obvious" formal shape

```
theorem constant_of_deriv_zero
  {a b : ℝ} (hab : a < b) {f : ℝ → ℝ}
  (hcont : ContinuousOn f (Icc a b))
  (hdiff : DifferentiableOn ℝ f (Ioo a b))
  (hderiv : ∀ x ∈ Ioo a b, deriv f x = 0) :
  ∀ x ∈ Icc a b, ∀ y ∈ Icc a b, f x = f y :=
  sorry
```

Reads off the informal sentence directly: continuity on $[a, b]$, differentiability on (a, b) , $f' = 0$ on (a, b) , conclude constancy.

No Mathlib lemma has this exact shape

Closest closed-interval lemmas in
`Mathlib.Analysis.Calculus.MeanValue`:

- `constant_of_derivWithin_zero`
- `constant_of_has_deriv_right_zero`
- `is_const_of_deriv_eq_zero` (no interval)

None accept "`DifferentiableOn + deriv = 0` on `Ioo a b`" as written. The alignment gap is in the **derivative notion**: at endpoints Mathlib uses `derivWithin / HasDerivWithinAt`, not `deriv / HasDerivAt`.

Worked Example: $f' \equiv 0 \implies f$ Constant (2/2)

Real Mathlib API

```
-- closed interval  $\impl$  derivWithin, not deriv
example
  {a b : ℝ} {f : ℝ → ℝ}
  (hdiff : DifferentiableOn ℝ f (Icc a b))
  (hderiv :  $\forall x \in \text{Ico } a \ b,$ 
    derivWithin f (Icc a b) x = 0) :
     $\forall x \in \text{Icc } a \ b, f \ x = f \ a :=$ 
    constant_of_derivWithin_zero
  hdiff hderiv
```

The lemma is `constant_of_derivWithin_zero`.
Same lesson as the LeanArchitect / Multivariate Taylor case at the end of Part IV: at endpoints, use the `Within` variants.

Four alignment shifts vs. draft

- `DifferentiableOn` on `Icc` (continuity inherited)
- `derivWithin`, not `deriv`
- condition on `Ico a b`, not `Ioo a b`
- conclusion pinned to $f(a)$, not symmetric

The "deriv = 0 $\impl$ constant" theorem has at least two real Mathlib shapes – both use the `Within` variants. **Neither** uses `deriv` or `HasDerivAt` directly.

Mathlib Does Not Start From Textbook Notation

Notation used by the Putnam B2 statement earlier:

$$\int x \text{ in } a..b, f x$$

comes from

`MeasureTheory.Integral.IntervalIntegral`. Not Riemann sums as primary definition; an oriented interval integral built on the measure-theoretic integral.

$$\begin{aligned} \int x \text{ in } a..b, f x \, \partial\mu \\ &= \int x \text{ in } \text{Ioc } a \, b, f x \, \partial\mu \\ &\quad - \int x \text{ in } \text{Ioc } b \, a, f x \, \partial\mu \end{aligned}$$

Orientation is part of the API:

$$\begin{aligned} \int x \text{ in } a..a, f x \, \partial\mu &= 0 \\ \int x \text{ in } b..a, f x \, \partial\mu \\ &= - \int x \text{ in } a..b, f x \, \partial\mu \\ \int x \text{ in } a..b, f x \, \partial\mu \\ &+ \int x \text{ in } b..c, f x \, \partial\mu \\ &= \int x \text{ in } a..c, f x \, \partial\mu \end{aligned}$$

Formal libraries often choose definitions for theorem engineering, not for matching first-semester textbook wording.

Bochner Integral: Lebesgue For Vector-Valued Functions

Concrete example: velocity $\rightarrow$ displacement

$\mathbf{v} : [0, T] \rightarrow \mathbb{R}^3$ – particle velocity in 3-space.

$$\int_0^T \mathbf{v}(t) dt \in \mathbb{R}^3$$

A **vector** integral. Integrate each coordinate Lebesgue-style; collect into one vector.

Bochner: same idea, any value type

$f : \mathbb{R} \rightarrow E, \int f dx \in E$. E = any space with "add" and "measure size" ($\mathbb{R}, \mathbb{R}^n, \mathbb{C}, \dots$).

$E = \mathbb{R} \Rightarrow$ Bochner = Lebesgue.

Nothing new for this course.

When Mathlib says `MeasureTheory.integral`, read "Lebesgue integral". The Bochner name signals the library kept the door open for vector / complex / matrix cases – one definition proved once, specialises to all.

The Underlying Integral Is Bochner Integral

`MeasureTheory.integral` is the Bochner integral.

```
MeasureTheory.integral
input:
  measure  $\mu$  on domain  $\alpha$ 
  function  $f : \alpha \rightarrow E$ 
  structure on  $E$ 
    sufficient for integration
output:
  element of  $E$ 
```

Not only real-valued functions; takes values in suitable normed vector spaces.

Common notations:

```
 $\int x, f \, x \, \partial\mu$ 
 $\int x \text{ in } s, f \, x \, \partial\mu$ 
 $\int x \text{ in } a..b, f \, x \, \partial\mu$ 
```

Behind ordinary-looking notation sits a hierarchy:

```
measurable space / measure
→ integrable functions
→ Bochner integral
→ set integral
→ interval integral
```

Formalization step one: locate the concept in the library.

Mathlib Also Has Riemann-Style Integrals

Riemann-style lives in `BoxIntegral`:

```
BoxIntegral.HasIntegral  
BoxIntegral.Integrable  
BoxIntegral.integral  
BoxIntegral.IntegrationParams.Riemann
```

Same framework hosts Henstock-Kurzweil and McShane via different `IntegrationParams`.

Textbook phrase → Mathlib object:

```
"the Riemann integral"  
  → BoxIntegral + IntegrationParams.Riemann  
"integral from a to b"  
  → intervalIntegral over MeasureTheory.integral
```

Bridge theorems connect the two under suitable hypotheses. Multiple definitions, one common proof interface.

What The Integral Case Teaches About Alignment

Template for the rest of the lecture:

informal phrase

"the integral of f from a to b "

formal choices

type of f

codomain structure

measure

interval convention

integrability condition

endpoint behavior

intended theorem shape

An informal mathematical statement is often a compressed reference to a whole formal design space.

The integral example was concrete. Part II abstracts it into a general checklist.

Part II. General Mechanism

From informal sentence to formal statement

The Statement Is The Object

Week 8 builds:

a formal theorem statement

(not a proof)

Two checks on a formal statement:

1. It typechecks in the formal language.
2. It expresses the intended mathematical meaning of the informal statement.

Dangerous autoformalization failures pass (1) but fail (2).

Example:

Informal: *Every subgroup of a cyclic group is cyclic.*

A type-correct but wrong formalization could:

- state the ambient group is cyclic in the conclusion instead of the subgroup;
- use the wrong formal definition of "cyclic".

Alignment Checklist

1. **Objects.** Variables, functions, sets, structures, parameters.
2. **Types.** Each object's type.
3. **Structures.** Algebraic, order, topological, metric, measure, category.
4. **Quantifiers.** Universal, existential, order.
5. **Assumptions.** Explicit vs convention-hidden.
6. **Definitions.** Which library definition corresponds.
7. **Notation.** Overloaded symbols disambiguated.
8. **Statement strength.** Too weak, too strong, just right.
9. **Edge cases.** 0, empty set, equal endpoints, boundaries.
10. **Back-translation.** Translate the formal back; does it still read as the original theorem?

This is the human version of statement validation. Modern systems (Part IV) automate parts of it.

Typechecking Is Not Semantic Alignment

A Lean statement can be well-typed and still be the wrong theorem.

Common failure modes

- **Too weak.** Only a special case.
- **Too strong.** Extra hypotheses, or sharper conclusion.
- **Wrong object.** Nearby but different definition.
- **Wrong scope.** Quantifier order swapped.

- **Wrong ambient structure.** Specialized to $\mathbb{R}$ instead of generic.
- **Wrong convention.** Index from 0 vs 1; open vs closed interval; left derivative vs derivative.

Formal statement checking is not enough. We also need statement alignment.

Part III. Worked Examples

Simple to research-level alignment problems

The First n Odd Numbers Sum To n^2

Informal:

The sum of the first n odd numbers is n^2 .

Hidden choices

- n is a natural number?
- "first n " means n terms, or up to n ?
- Odd numbers as $2i + 1$ or $2i - 1$?
- Index $0..n - 1$ or $1..n$?
- Sum over `Finset.range`, list, recursion?
- Equality in `Nat`, `Int`, `Real`?

A possible Lean statement:

```
example (n : Nat) :  
  (∑ i in Finset.range n, (2 * i + 1))  
    = n ^ 2 := by  
  ...
```

This statement has already made these choices:

- `n : Nat`;
- zero-based indexing;
- first n terms are $1, 3, \dots, 2n - 1$;
- finite sum over `Finset.range`;
- equality in `Nat`.

Set Distributivity

Informal:

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$$

Hidden choices

- Universe type α ?
- A, B, C are `Set  $\alpha$` ?
- Extensional set equality?
- $\cap, \cup$ as set ops or lattice ops?

```
variable { $\alpha$  : Type*} (A B C : Set  $\alpha$ )
```

```
example :
```

```
  A  $\cap$  (B  $\cup$  C)  
    = (A  $\cap$  B)  $\cup$  (A  $\cap$  C) := by  
  ...
```

Even this simple example forces

- a universe α
- objects typed as `Set  $\alpha$`
- set lattice notation
- extensional equality

The Inverse Of A Product

Informal:

The inverse of a product is the product of the inverses in reverse order.

Hidden choices

- Group, monoid-with-inverses, division ring, or category?
- Multiplication commutative or not?
- Conclusion $(ab)^{-1} = b^{-1}a^{-1}$?
- In a commutative setting, reverse order is hidden; otherwise substantive.

```
variable {G : Type*} [Group G]

example (a b : G) :
  (a * b)-1 = b-1 * a-1 := by
  ...
```

Natural language says "product" without naming the algebraic structure. Formal language cannot use multiplication until the structure is fixed.

Every Subgroup Of A Cyclic Group Is Cyclic

Informal:

Every subgroup of a cyclic group is cyclic.

Hidden choices

- Ambient group G .
- Multiplicative vs additive notation.
- "Cyclic" as a predicate on groups, subgroups, or modules?
- Subgroup as bundled `Subgroup G`?
- Subgroup cyclicity uses the inherited group structure?

```
variable {G : Type*} [Group G]

-- sketch shape; final mathlib API
-- packages this differently:
∀ H : Subgroup G,
  IsCyclic G → IsCyclic H
```

"Subgroup" in natural language refers both to a subset-like object and to a group in its own right. Formalization must choose the bundled representation that exposes the needed structure.

Continuous Functions On $[a, b]$ Attain A Maximum

Informal:

A continuous function on $[a, b]$ attains a maximum.

Hidden choices

- $a \leq b$ assumed?
- $f : \mathbb{R} \rightarrow \mathbb{R}$, or $f : X \rightarrow \mathbb{R}$?
- `Continuous f` vs `ContinuousOn f (Set.Icc a b)`?
- "Attains a maximum" means $\exists x \in [a, b], \forall y \in [a, b], f(y) \leq f(x)$?
- Is this really compact-set extreme value?

$$\begin{aligned} &\exists x \in \text{Set.Icc } a \ b, \\ &\forall y \in \text{Set.Icc } a \ b, \ f \ y \leq f \ x \end{aligned}$$

A formal statement here specifies:

- domain set: `Set.Icc a b`;
- codomain order: $\leq$ on $\mathbb{R}$;
- continuity predicate: `ContinuousOn`;
- compactness source: closed bounded interval.

A common theorem name often hides compactness, order, and domain restriction.

$\text{Gal}(E/\mathbb{Q}) \cong Q_8$: One Statement, Several Formal Shapes

Informal:

Let $\alpha = \sqrt{(2 + \sqrt{2})(3 + \sqrt{3})}$ and $E = \mathbb{Q}(\alpha)$.
Show $\text{Gal}(E/\mathbb{Q}) \cong Q_8$.

Hidden choices

- α as `Real.sqrt ...`;
- E as `IntermediateField.adjoin  $\mathbb{Q}$  { $\alpha$ }`;
- $\text{Gal}(E/\mathbb{Q})$ as `$\mathfrak{r}E \approx_a[\mathbb{Q}] \mathfrak{r}E$` ;
- Q_8 as `QuaternionGroup 2`;
- $\cong$ as `$\approx^*$` (Type, not Prop).

```
-- Shape A: explicit MulEquiv (def, not theorem)
def gal_iso_Q8 :
  ( $\mathfrak{r}E \approx_a[\mathbb{Q}] \mathfrak{r}E$ )  $\approx^*$  QuaternionGroup 2 := by
  sorry

-- Shape B: Galois-group predicate
theorem q8_is_galois_group
  [MulSemiringAction (QuaternionGroup 2)  $\mathfrak{r}E$ ] :
  IsGaloisGroup
    (QuaternionGroup 2)  $\mathbb{Q}$   $\mathfrak{r}E$  := by
  sorry

-- Shape C: finite Galois group object
noncomputable def q8_as_finGaloisGroup_iso :
  Efin.finGaloisGroup
     $\equiv$  FiniteGrp.of (QuaternionGroup 2) := by
  sorry
```

Three Shapes, Three API Paths

Shape A – MulEquiv

Direct group isomorphism.

$(\mathbb{1}E \simeq_a [\mathbb{Q}] \mathbb{1}E) \simeq^* \text{QuaternionGroup } 2$

Closest to the informal $\cong$. Must use `def` (the type is `Type`, not `Prop`).

Shape B – IsGaloisGroup

Q_8 acts faithfully on E with fixed field $\mathbb{Q}$.

`IsGaloisGroup (QuaternionGroup 2)  $\mathbb{Q}$   $\mathbb{1}E$`

Mathlib bridges to `MulEquiv` via `IsGaloisGroup.mulEquivAlgEquiv`.

Shape C – FiniteGrp

Package E as `FiniteGaloisIntermediateField`.

`Efin.finGaloisGroup  $\cong$  FiniteGrp.of (QuaternionGroup 2)`

"Finite Galois" lives inside the object, not as a separate hypothesis.

Three reasonable formal targets for one informal isomorphism. Choice of shape changes available API and downstream proof routes.

Does $\sum_{n=1}^{\infty} \frac{1}{n}$ Converge?

Hidden commitments

NL	hidden choice
$\sum_{n=1}^{\infty}$	partial-sum vs limit object
$\frac{1}{n}$	$\mathbb{Q}$ -, $\mathbb{R}$ -, $\mathbb{C}$ -valued?
"converge"	classical limit, or $= +\infty$?

Three Lean shapes (equivalent in $\mathbb{R}$)

```
∃ L, Tendsto (fun N => ∑ k in range N, 1/(k+1)) atTop (ℕ L)
Summable (fun n : ℕ => (1:ℝ)/(n+1))
HasSum (fun n : ℕ => (1:ℝ)/(n+1)) L
```

Answer (Euler, 1734)

$\sum 1/n = +\infty$ via integral comparison:
 $\int_1^{N+1} dx/x \leq \sum_{n=1}^N 1/n$, LHS = $\ln(N+1) \rightarrow \infty$.

Alternating: opposite story

$\sum (-1)^{n-1}/n = \ln 2$, converges **conditionally**.
Lebesgue/Bochner refuses it (absolute-integrability rule from the Lebesgue primer).

****Trap:**** $\neg \text{Summable } (1/n) \checkmark$ and $\text{tsum } (1/n) = \top \checkmark$ – but the second uses $\mathbb{R} \geq 0 \infty$, not $\mathbb{R}$.
Pattern-matching "diverges" $\rightarrow "= \infty"$ picks wrong target.

Part IV. Modern Autoformalization Systems

Mechanisms aimed at the alignment problem

What Modern Systems Are Trying To Fix

*The formal statement **compiles**,
and is **not** the intended theorem.
This is the central failure mode every Part IV paper attacks.*

Five failure shapes seen in the wild

- **Free-variable swap** – `pi` $\neq$ `Real.pi`
- **Default-argument trap** – `(f := ...)` is a Lean default, not an asserted equation
- **Wrong object** – `Continuous` vs `ContinuousOn`
- **Junk-value collapse** – `deriv f x = 0` even when f not differentiable (the deriv trap from Part I)

Each paper attacks one piece

Cog	Paper
Stronger acceptance	Rethinking – BEq
Library grounding	Rethinking, Aria, LoC
Self-critique	ReForm
No gold reference	Roundtrip
FL $\rightarrow$ NL infra	Herald / LeanSearch
Project scale	LeanArchitect

Rethinking: Typecheck Is Too Weak

*Lean accepts the statement
does **not** imply
the statement is the intended theorem.*

The fix: BEq

Bidirectional Extended definitional Equivalence:

both directions $S_{\text{gen}} \leftrightarrow S_{\text{ref}}$ must derive under a bounded set of rewrites.

Single direction can silently weaken the claim.

Both directions cannot.

On 200 expert-labeled pairs

Metric	Prec.	Acc.
Typecheck	35	35
BLEU	63	69
LLM judge	71	83
BEq	100	91

BEq's recall is 73 % – the precision-100 zone. The gap above 73 % is where Aria / ReForm / Roundtrip do their work.

Rethinking: Retrieval Beats Generation

*The bottleneck is not how well the LLM writes Lean.
It is **which Mathlib objects sit in the LLM's context.***

Build an informalized Mathlib

243,797 formal objects + 139,933 theorems

- ↓ bottom-up informalization
- ↓ along the dependency graph

each object carries:
formal sig + NL description
+ list of dependencies

Then put the retrieved deps in the prompt

Old prompt:

"Translate this theorem to Lean."

RAutoformalizer prompt:

"Available Mathlib objects:
Complex.differentiableOn, ...
Now write the statement."

Con-NF BEq@8

baseline (no retrieval):	4.6 %	
+ retrieval:	16.9 %	+12 pp
+ ORACLE deps (upper bound):	~35 %	+18 pp

The retrieval bump is half the climb to oracle. Plain semantic search misses the right Mathlib name two-thirds of the time (Recall@5 $\approx$ 35 % on ProofNet, 24 % on Con-NF).

Aria: Concept Dependency Graph Before Lean Code

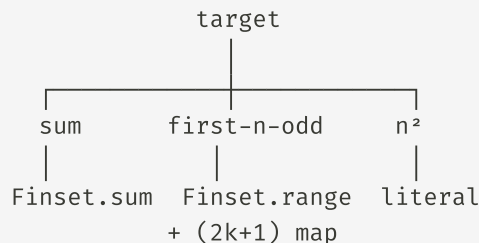
*Never write Lean from NL directly.
First decompose NL into a **concept graph**;
ground the leaves in Mathlib;
synthesize the missing inner nodes.*

The GoT pipeline (6 stages)

NL statement

- concept dependency graph
- ground leaves via LeanSearch
- bottom-up synthesis of concepts missing from Mathlib
- compiler-in-the-loop reflection
- Lean theorem statement

Worked example: "first n odd numbers sum to n^2 "



Every leaf grounds in Mathlib; inner nodes synthesized.

Headlines: ProofNet 68.5 %, FATE-H 71 %, homological conjectures 42.9 %. Ablations are decisive: **remove RAG → 0 %** on conjectures; **remove GoT → 6/14 → 1/14**. Both retrieval and graph structure are load-

AriaScorer: Term-Level Grounding

Question

A generated definition **compiles** – how do we know it is the intended **mathematical concept**?

Mechanism

For each term in the candidate Lean, `jixia` retrieves from informalized Mathlib: name, type, def body, informal name + description. A judge compares assumptions/conclusions in this *grounded* context, not on surface strings.

Traps surface checkers miss

UFD / IsDomain. Surface checker flags missing ``IsDomain``; AriaScorer sees ``UniqueFactorizationMonoid`` already inherits it.

QuaternionGroup 1 vs 2. ``QuaternionGroup n`` has order $4n$. Surface match accepts either; AriaScorer reads the index – wrong for Q_8 .

Makes Aria **self-correcting**: a compiled-but-wrong synthesis triggers another GoT pass with the failed term marked for re-grounding.

LoC-Decomp: A Template That Forces Commitments

*Replace the one-shot theorem header
with a **7-section template**.
Each section forces a commitment NL hides.*

The seven sections

- | | |
|-------------------------|-------------------------|
| 1. imports | ← which Mathlib API |
| 2. aux types | ← what kinds of objects |
| 3. mathematical systems | ← the unknowns |
| 4. aux functions | ← helper notation |
| 5. constraints | ← hypotheses |
| 6. solution constraints | ← what "the answer" is |
| 7. final statement | ← the theorem |

Worked: "find all n with n^2+n+1 prime"

One-shot LLM:

```
theorem foo (n : ℕ) :  
  Nat.Prime (n2+n+1) ↔ n = 1
```

– wrong: is $n = 0$ allowed? "find all" = set,
predicate, or iff?

Template forces:

```
answer : Set ℕ  
answer = {n | n ≥ 1 ∧ Nat.Prime (n2+n+1)}  
theorem foo : answer = {1}
```


LoC-Decomp: ASCC + Numbers

*ASCC = Automatic Semantic Consistency Check.
Back-translate **per template section**, not whole file.
Discrepancies localise to a Lean line.*

Four-cell diagnostic

Compile	ASCC	Diagnosis
✓	✓	alignment evidence
✓	✗	wrong theorem proved
✗	✓	math right, API wrong
✗	✗	broken at both layers

The (✓ **compile**, ✗ **ASCC**) cell is the failure mode
NL → FL systems bleed through.

DeepSeek-V3 + LoC-Decomp pass rate

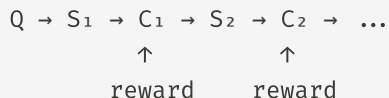
	MATH-500	miniF2F
raw LLM	~35 %	—
+ template	63 %	77 %
+ ASCC loop	84 %	90 %
Δ from loop	+21 pp	+13 pp

ASCC checker itself on 150 expert-labeled pairs: P
= 0.90, R = 0.82, F1 = 0.86.

The loop is doing as much work as the template: structured decomposition alone leaves a 20-pp gap; only the per-segment feedback closes it. ReForm (next) moves the same loop *inside* the model via RL.

ReForm: Reflection Loop + PBSO Reward

Architecture



Each C_t should cite Q 's structure, name a missing commitment, surface an ambiguity.

Reward split

$$\mathcal{L} = -\mathbb{E}[R_{\text{task}} + \lambda \sum_t R_{\text{aux}}(t)]$$

- R_{task} : terminal – S_n passes Lean + semantic check
- $R_{\text{aux}}(t)$: per-turn – quality of C_t as diagnosis

Without R_{aux} : model rubber-stamps and copies S_1 to S_n .

Shallow vs specific critique

****Shallow:**** "Syntax valid; matches problem." → $R_{\text{aux}} \approx 0.1$

****Specific:**** "Q is conditional 'if $F=f$ then ...'. S_1 never introduces F as antiderivative. Missing: `HasDerivAt F (f x) x`; integrability of f . `HasDerivAt` vs `HasDerivWithinAt` unspecified." → $R_{\text{aux}} \approx 0.85$

ReForm: Numbers + Audit Of "Gold" Benchmarks

Headline (semantic-consistency %)

Model	miniF2F	ProofNet	Putnam	AIME
Baseline best	73	48	36	22
ReForm-32B	91	70	62	67
Δ	+18	+23	+27	+45

Pattern: **harder benchmark, larger lift**. Average +22.6 pp.

The audit finding

Running ReForm's ConsistencyCheck (859 items) on **human-written** gold benchmarks:

miniF2F: **16.4 %** have semantic errors.
ProofNet: **38.5 %** have semantic errors.

Three caught examples:

- $\sqrt{11}$ rewritten as **11** in Lean
- $\deg p \leq 80 \rightarrow \deg p < 80$
- "find X " baked into "prove $X = c$ "

If gold references themselves carry 16–39 % errors, "compare to reference" is not a clean ground truth.

Roundtrip (next) drops the gold-reference assumption entirely.

Roundtrip: Verification Without A Gold Target

The roundtrip

```
x      informal
y_orig = formalize(x)      T1
x'     = informalize(y_orig) T2
y_rt   = formalize(x')     T3

check  y_orig ≡ y_rt
```

Drift in T1, T2, or T3 if they disagree.

Scoped repair

Diagnosis	Repair
T1 broke	re-formalize, T2 & T3 re-run
T2 broke	only T3 re-runs
T3 broke	re-formalize y_{rt}

The only Part IV mechanism that operates without any gold reference. Everything else implicitly compares to *something*.

Three debug questions

1. T1: first Lean statement wrong? 2. T2: back-translation misleading? 3. T3: second formalisation unstable?

Parallel to LoC-Decomp

Both **localise the failure**: LoC by template segment, Roundtrip by pipeline stage.

Roundtrip: Numbers + Wrong Fixed Points

Statute-code accuracy (Z3 equivalence)

Texas Transportation Code / Parks & Wildlife:

No repair:	45 / 66 %
Claude + scoped repair:	85 / 86 %
Best (GPT + Claude diag):	91 / 95 %

The diagnoser is the lever

GPT default prompt blames T1 in **83 % / 97 %** of cases – nearly always. A non-sequential prompt drops to **9 % / 16 %**, and end-to-end accuracy rises. **Same models, only the diagnoser prompt changed.**

Wrong fixed point

If T2 drops the **same** hypothesis as T1, roundtrip stabilises on a wrong $y_{\text{orig}} \equiv y_{\text{rt}}$. Mitigation: multiple T2 paraphrase paths.

Why statute, not Lean

Z3 decides FOL equivalence; Lean equivalence often needs Mathlib lemmas. Same idea, harder checker.

Reach for Roundtrip when no gold target exists – Mathlib PRs, novel math, biomedical ontology. The wrong-fixed-point caveat is honest, not fatal.

Why The Reverse Direction Matters: Formal $\rightarrow$ Informal

Four uses for FL $\rightarrow$ NL

1. Training data. Mathlib4 has ~200 K formal decls, ~0 paired NL. Herald produces 44 k pairs – all by back-translation.

2. Retrieval. LeanSearch, Aria, ReProver index *informalized* Mathlib, not raw source.

3. Verification. ReForm, Roundtrip, Herald validator all back-translate $S \rightarrow Q'$ and compare $Q \approx Q'$. No FL $\rightarrow$ NL $\rightarrow$ no check.

4. Documentation. 1.5 M lines of Mathlib become navigable. Same component powers docs, errors, tutorials.

FL $\rightarrow$ NL is **not symmetric** with NL $\rightarrow$ FL. It is the prerequisite that makes the rest of Part IV possible.

Herald: Mathlib4 → Natural-Language Dataset

Pipeline

Mathlib4 → jixia → informalize → Herald dataset + Translator

Scale: 580 k statements, 44 k NL↔FL theorem pairs, 45 k proof pairs.

Same LLM, same statement – the difference is **structured metadata fed to the prompt**, not model capacity.

Set.Ico walkthrough

```
theorem foo (a b c : ℝ) :  
  c ∈ Set.Ico a b ↔ a ≤ c ∧ c < b
```

Naive: "c belongs to Ico from a to b iff $a \leq c$ and $c < b$." – identifier leaks.

Herald: "c lies in the left-closed right-open interval $[a, b)$ iff $a \leq c < b$."

LeanSearch: Informalized Mathlib As A Service

*Herald's dataset becomes a **query service**.
NL question in → relevant Mathlib declaration out.*

How it's used

```
NL: "continuous function on  
    a compact set attains  
    a maximum"  
↓ embedding model  
   trained on Herald pairs  
↓ nearest-neighbour over  
   informalized Mathlib  
↓ Mathlib decls:  
   IsCompact.exists_isMaxOn  
   ContinuousOn.isCompact_image
```

Downstream

Consumer	Use
LeanSearch	NL → Mathlib API search
Loogle	type-pattern search using NL labels
Aria / ReProver	retrieval inside NL → FL

Deployed at leansearch.net.

FL → NL is **not symmetric** with NL → FL. It is the **upstream infrastructure** behind every "RAG over Mathlib" pipeline in Part IV.

Herald Translator: Numbers + Two Case Studies

Coverage by domain

miniF2F-test	96.7 %
internal graduate text	23.5 %
College CoT	16.0 %

Contest math: nearly solved. Graduate math: much harder.

Validation pipeline

```
formal candidate
  → Lean compile check
  → back-translate to NL
  → NLI check vs original
```

Case 1: quantifier order ($\forall x \exists \delta$ vs $\exists \delta \forall x$)

" f continuous on X " $\rightarrow$ two non-equivalent Lean shapes:

```
 $\forall x, \forall \epsilon, \exists \delta, \forall y, \dots$  -- continuity
 $\forall \epsilon, \exists \delta, \forall x, \forall y, \dots$  -- uniform continuity
```

Heine–Cantor: B follows from A on compact X ; strictly stronger otherwise. Moving x past $\exists \delta$ silently rewrites the source.

Case 2: Four Lemma (mono half)

Lean header full of `ComposableArrows.app'  $\varphi$  i` ... and index proofs. Faithful NL must restore "*morphism between exact chains; if φ_0 epi and φ_1, φ_3 mono, then φ_2 mono*" – without unfolding the index machinery.

What Makes Formal → Informal Hard: Three Tax

1. Naming tax

Mathlib identifiers = encyclopaedic labels.

```
Set.Ico → [a,b)
StrictMono → strict inc.
IsCompact → compact
Mono/Epi → mono/epi
```

Need a learned naming map.

2. Type-class tax

Which implicit args surface?

- `[Abelian C]` → yes
 - `[DecidableEq G]` → no
 - `[CommRing R]` → yes
 - universes → no
- Heuristic, per Mathlib design.

3. Surface tax

- chain $a \leq c \wedge c < b \rightarrow a \leq c < b$
- bracket notation
- `mono_of_...` → "Four Lemma"
- $\mathbb{R}$ vs `Real`

Three taxes independent. Naming alone leaves `DecidableEq` noise; type-class alone leaves un-chained algebra; surface alone leaves leaking identifiers.

Beyond Theorem Headers: Three Levels Of FL → NL

Level 1: statement

```
ContinuousOn f s → IsCompact s  
  → IsCompact (f '' s)
```

→ "Continuous on compact
⇒ compact image."

Herald: 97 % miniF2F, 16 %
graduate.

Level 2: proof

```
intro ε hε  
obtain {δ, hδ, h} := ...  
calc |f y - f x| < ε/2 ...
```

→ NL prose with proof-state
context at every step.

Lean-STaR partial; mostly
open.

Level 3: definition

```
class CompleteLattice α extends ...  
  sSup : Set α → α  
  le_sSup ...
```

→ "Lattice with arbitrary
suprema."

Mathlib docstrings, hand-
written; no scalable
generation.

Most Part IV "autoformalization needs autoinformalization" claims assume **Level 1**. Levels 2 and 3 reopen the alignment problem in harder forms.

LeanArchitect: @[blueprint] – One Source Of Truth

Big Lean projects keep a LaTeX blueprint plus Lean code.

*Two artifacts drift; **proof agents trip over the drift**.*

LeanArchitect makes Lean the single source – LaTeX is derived.

The Lean attribute

```
@[blueprint "thm:add-comm"  
  (statement := /-- Addition is  
                  commutative. -/  
    (uses := [def:nat, lem:zero-add]))  
theorem MyNat.add_comm ...
```

Holds the LaTeX label, NL statement, declared deps. The extractor reads constants actually used and emits `\leanok`, `\uses{ ... }`, `\notready` automatically.

Drift modes auto-sync kills

- stale prose vs Lean header → edited together
- missing/wrong `\uses{ ... }` → extractor cross-checks
- wrong `\leanok` → computed, not asserted
- divergent edits → only one file to edit

Caveat. Auto-sync $\neq$ semantic equivalence. @[blueprint] closes *metadata* drift. Whether the NL statement

LeanArchitect Case: Multivariate Taylor And derivWithin

Workflow

GPT-5 drafts blueprint + sorrys → LeanArchitect graph → Aristotle proves leaves → human inspects failures.

What happened

~40 nodes; **37 discharged**; **3 stuck**. All on derivatives *on the segment* $[a, b]$.

The bug – not a proof problem

Lean used ``deriv`` for a function on segment $[a, b]$. Correct: ``derivWithin (lcc a b)``. Outside the segment ``deriv`` returns junk; the statement was never the segment-restricted theorem.

With the graph: trace upstream → fix the statement, not the prover.

****Same alignment trap, three scales**** – Part I's ``deriv`` junk value, Part III's ``ContinuousOn`` vs ``Continuous``, here ``deriv`` vs ``derivWithin`` at project scale.

Part IV Synthesis

Better acceptance

What the statement *means*, not just compiles.

- BEq (precision 1.00)
- AriaScorer term grounding
- ConsistencyCheck audit

Library grounding

Autoformalization = retrieval + generation.

- RAutoformalizer (+12 pp)
- Aria GoT pipeline
- Herald + LeanSearch (infra)

Feedback + scale

Alignment is a process.

- LoC ASCC loop (+20 pp)
- ReForm + PBSO (+22.6 pp)
- Roundtrip (no gold)
- @[blueprint] at scale

Seven papers, one diagnosis: ****typecheck is the start line, not the finish line****. The Part V activity is a hand-run miniature of BEq + AriaScorer + ASCC + Roundtrip.

Part V. Alignment Template

Fields a formal statement must pin down

Fields A Formal Statement Must Pin Down

Informal statement:

Objects:

Types:

Structures / typeclasses:

Definitions from library:

Quantifiers:

Assumptions:

Conclusion:

Edge cases:

Back-translation:

The deliverable is not a proof. It is a formal statement whose every field above has been committed to deliberately.

Sample Informal Claims For Alignment

A. The inverse of a product is the product of inverses in reverse order.

B. A continuous function on $[a, b]$ attains a maximum.

C. Every subgroup of a cyclic group is cyclic.

D. The first n odd numbers sum to n^2 .

E. The indefinite integral of f is F .

Three possible formal targets: derivative-side predicate, derivative-side set, or a specific definite-integral-with-variable-upper-limit. Each gives a different theorem.

Alignment Audit: Six Questions For Any Formal Statement

Six questions to check a formal statement S against an informal claim Q :

1. If S is true, does Q follow?
2. If Q is true, does S follow?
3. Was an assumption silently added that wasn't there in Q ?

4. Was a needed assumption dropped?
5. Was the object swapped (e.g. ambient group instead of subgroup, `deriv` instead of `derivWithin`)?
6. Does a back-translation of S still sound like Q ?

"Yes" on (1) and (2), "no" on (3)-(5): the minimum bar for semantic alignment. Typecheck does not give this for free.

Part VI. Mathematical Sidebars

Harmonic series, primes, and density

Standalone math; not assessed.

Leonhard Euler (1707–1783)



Handmann pastel, 1753

Highlights

- Proved $\sum 1/n = \infty$
- Euler product: $\zeta(s) = \prod_p (1 - p^{-s})^{-1}$
- First analytic proof of infinitely many primes (via $\sum 1/p$ diverging)
- Standardised $\sum, e, i, \pi, f(x)$

Euler's identity $\sum_p \ln \frac{1}{1-1/p} = \sum_n 1/n$ is one line that hides type choice, convergence condition, and infinite-product definition. The first alignment question is: *what does this $\sum$ actually mean?*

Harmonic Series: $\sum \frac{1}{n}$ Diverges

Integral comparison

$$\sum_{n=1}^N \frac{1}{n} \geq \int_1^{N+1} \frac{dx}{x} = \ln(N+1)$$

As $N \rightarrow \infty$, the right-hand side $\rightarrow \infty$, so the partial sums $\rightarrow \infty$ as well.

Cauchy's blocking proof

Group terms by powers of 2:

$$\sum = 1 + \frac{1}{2} + \underbrace{\left(\frac{1}{3} + \frac{1}{4}\right)}_{\geq 1/2} + \underbrace{\left(\frac{1}{5} + \dots + \frac{1}{8}\right)}_{\geq 1/2} + \dots$$

Infinitely many blocks each $\geq \frac{1}{2} \Rightarrow$ total is ∞ .

Numerical taste of the divergence

N	$\sum_{n=1}^N 1/n$
10	2.929
100	5.187
1 000	7.485
10 000	9.788
10^9	≈ 21.30

Growth is logarithmic. Euler's constant $\gamma = \lim(H_N - \ln N) \approx 0.5772$ – never proved irrational, still open.

The series diverges, but **slowly**. This is the heart of *conditional* rearrangements (Riemann's rearrangement theorem) – slow divergence is what

Alternating Harmonic: $\sum \frac{(-1)^{n-1}}{n} = \ln 2$

Convergence (Leibniz)

a_n positive, decreasing, $\rightarrow 0 \Rightarrow \sum (-1)^{n-1} a_n$ converges. $a_n = 1/n$ satisfies all three.

Limit value

From $\ln(1+x) = x - x^2/2 + x^3/3 - \dots$ at $x = 1$ (Abel summation):

$$\sum_{n=1}^{\infty} \frac{(-1)^{n-1}}{n} = \ln 2 \approx 0.6931$$

Conditional, not absolute

$$\sum |(-1)^{n-1}/n| = \sum 1/n = \infty.$$

****Riemann rearrangement:**** conditional convergence $\Rightarrow$ you can rearrange to any real, $+\infty$, or divergent. The value $\ln 2$ depends on order.

****Lebesgue connection:**** Lebesgue/Bochner sum on $\mathbb{N}$ requires $\sum |a_n| < \infty$. Alternating harmonic = canonical case where Riemann accepts a value Lebesgue refuses.

Euler's Proof: Infinitely Many Primes

Euler product (1737)

For $s > 1$, $\zeta(s) = \sum 1/n^s = \prod_p (1 - p^{-s})^{-1}$
by unique factorization.

As $s \rightarrow 1^+$

LHS: $\zeta(s) \rightarrow \sum 1/n = \infty$, so product diverges:
 $\prod_p (1 - 1/p)^{-1} = \infty$.

Take logs

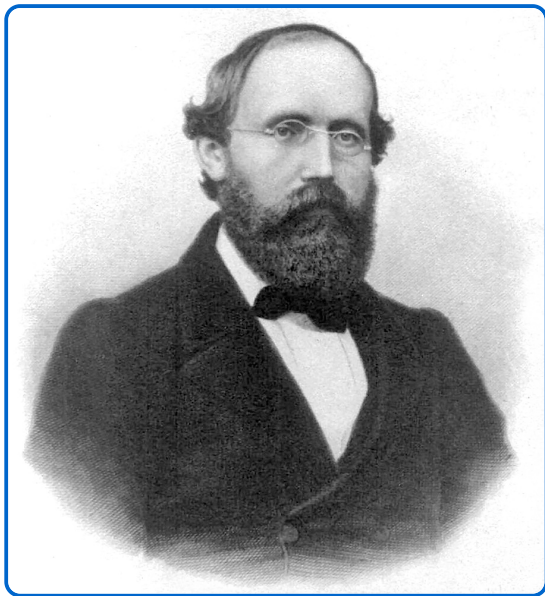
$$\sum_p \ln \frac{1}{1-1/p} = \infty$$

Using $\ln \frac{1}{1-x} = x + x^2/2 + \dots$, the tail
 $\sum_p (1/2p^2 + \dots)$ converges (bounded by
 $\sum 1/n^2$), so:

$$\boxed{\sum_p \frac{1}{p} = \infty}$$

Finitely many primes $\Rightarrow$ finite sum. So infinitely many – and $\sum 1/p$ diverges. Compare $\sum 1/n^2 < \infty$: primes are "denser than squares".

Bernhard Riemann (1826–1866)



Portrait c. 1863

Highlights

- Extended $\zeta(s)$ to a **meromorphic function** on $\mathbb{C}$ with one simple pole at $s = 1$
- 1859 memoir linked ζ 's zeros to prime distribution
- **Riemann Hypothesis:** zeros on $\Re(s) = 1/2$ – open since
- **Identity theorem:** uniqueness of analytic continuation
- Riemann integral, curvature, mapping theorem

Next slide: " ζ has a pole at $s = 1$ " – one Lean statement invoking the whole continuation + uniqueness apparatus.

Meromorphic Functions And The Identity Theorem

What "meromorphic" means

$f : U \subseteq \mathbb{C} \rightarrow \mathbb{C}$ is meromorphic on a connected open U if f is holomorphic except at isolated **poles** – finitely many negative-index terms in the Laurent expansion near each singular point.

Equivalently: $f = g/h$ with g, h holomorphic, $h \not\equiv 0$.

Riemann's zeta

$\zeta(s)$ is meromorphic on $\mathbb{C}$ with a **single simple pole** at $s = 1$, residue 1 – the analytic shadow of $\sum 1/n = \infty$.

Identity theorem (uniqueness)

Meromorphic f, g on connected open U agreeing on a set with a limit point in $U \Rightarrow f = g$ on U .

So **the** meromorphic extension is unique. Without it, $\zeta(-1) = -1/12$ would not be a meaningful claim.

" ζ extends to a meromorphic function on $\mathbb{C}$ with a simple pole at $s = 1$ " hides 5 Lean commitments: complex-valued function / holomorphic open subset / order of singularity / uniqueness /

Peter Gustav Lejeune Dirichlet (1805–1859)



Public-domain portrait

Highlights

- **Dirichlet's theorem (1837):** infinitely many primes in every coprime AP
- **Dirichlet characters χ + L-functions $L(s, \chi) = \sum \chi(n)/n^s$** – the proof engine
- **Dirichlet density** – works where natural density doesn't
- Pigeonhole principle, Dirichlet series, BC, function $\chi_{\mathbb{Q}}$

Dirichlet's theorem is the first non-trivial result on prime *distribution*, and the proof launched analytic number theory.

Dirichlet's Theorem And Density

Dirichlet (1837)

For coprime a, q , the AP $a, a + q, a + 2q, \dots$ contains **infinitely many primes**.

Equidistribution

$\varphi(q)$ coprime residue classes; primes uniformly distributed:

$$d_{\text{Dir}}(\{p : p \equiv a \pmod{q}\}) = \frac{1}{\varphi(q)}$$

The mod- q analogue of PNT.

Two density definitions

Natural: $d_{\text{nat}}(S) = \lim_{x \rightarrow \infty} \frac{\#\{p \in S, p \leq x\}}{\pi(x)}$

Dirichlet: $d_{\text{Dir}}(S) = \lim_{s \rightarrow 1^+} \frac{\sum_{p \in S} p^{-s}}{\sum_p p^{-s}}$

d_{nat} existing $\Rightarrow d_{\text{Dir}}$ equals it. ****Converse fails**** – wrong formal definition silently changes the theorem.

Why These Three Topics Sit Together
